Identify memory leak in this code

```
#include<iostream>
#include<cstring>
using std::cout;
using std::endl;
class Str {
    int len;
    char *ptr;
public:
    Str(const char *ptr) {
        len = strlen(ptr);
        this->ptr = new char[len];
        strcpy(this->ptr, ptr);
    }
    void print() {
        cout << ptr << endl;
    }
};
int main() {
    for(int i = 0; i < 5; i++) {
        Str st1(" Hello world, this is just an example!");
        cout << i;
        st1.print();
    }
    return 0;
}
```

```
#include<iostream>
#include<cstring>
using std::cout;
using std::endl;
class Str {
    int len;
    char *ptr;
public:
    Str(const char *ptr) {
        cout << "Constructor called" << endl;
        len = strlen(ptr);
        this->ptr = new char[len + 1];
        strcpy(this->ptr, ptr);
    }
    void print() {
        cout << ptr << endl;
    }
    ~Str() {
        cout << "Destructor called" << endl;
        delete[ ] ptr;
    }
};
int main() {
    for(int i = 0; i < 5; i++)
    {
        Str st1(" Hello world, this is just an example!");
        cout << i; st1.print();
    }
    return 0; }
```

```
Constructor called
0 Hello world, this is just an example!
Destructor called
Constructor called
1 Hello world, this is just an example!
Destructor called
Constructor called
2 Hello world, this is just an example!
Destructor called
Constructor called
3 Hello world, this is just an example!
Destructor called
Constructor called
4 Hello world, this is just an example!
Destructor called
```

Destructor

- Destructor function cannot have parameters
- There exists no mechanism by which to pass arguments to an object that is being destroyed
- Destructor cannot be overloaded

Destructor

- Objects are destroyed in the reverse order of their creation
- **Can there be more than one destructor in a class?**
- Answer : no
- **When do we need to write a user-defined destructor?** If we do not write our own destructor in class, compiler creates a default destructor for us. The default destructor works fine unless we have dynamically allocated memory or pointer in class. When a class contains a pointer to memory allocated in class, we should write a destructor to release memory before the class instance is destroyed. This must be done to avoid memory leak.

Destructors

- **A destructor is a special member function that is called just before the lifetime of an object ends.**
  - **Object's lifetime ends only after its destructor finishes its execution. The execution of a destructor is the last thing in the lifetime of an object.**
  - **The destructor is called whenever an object's lifetime ends, which includes**
    - **Program termination, for objects with static storage duration (static and global)**

```
static MyClass globalObj;) // Destructor called when program ends
```

    - **End of scope, for objects with automatic(Local) storage duration**

```
void func() {
    MyClass localObj;
    // Destructor called at end of func()
}
```

- **Delete-expression, for objects with dynamic storage duration**

```
MyClass* ptr = new MyClass();
delete ptr; // Destructor called here
```

- **The purpose of the destructor is to free the resources (e.g. Memory, files, etc.) that the object may have acquired during its lifetime.**
  - For example, if memory is allocated dynamically for object data members, it must be released before the lifetime of the object ends. Destructor is the obvious choice to do it.

Destructors

- **If no user-declared destructor is provided for a class then the compiler will provide a default destructor**

- **The name of the destructor is the same as the name of the class but is preceded by a tilde**
  - **~class_name();**
- **Like the constructor, the destructor does not return any value, hence no return statement in the destructor body**
- **Unlike the constructor, the destructor does not have any parameters, hence it can not be overloaded and there can only be one destructor per class**
- **Destructor can be defined within class definition or outside class definition**
  - If it is defined outside the class definition then like other member functions, the name of the class and scope resolution operator are used for full qualification

- **Auto and global objects are destroyed in reverse order than order of their creation**

```
#include <iostream>
using namespace std;
int count=0;
class test
{

   public:
     test( )
     {
      count++;
            cout<<" constructor for object
number"<<count<<"\n";

     }
     ~test()
     {
     cout<<"destructor for object
number"<<count<<endl;
     count--;

     }
 };
```

```
int main()
 {
  cout<<"inside main\n";
  cout<<"creating object T1\n";
  test T1;
  {
  //Block
  cout<<"creating two more object in
block T2 and T3\n";
  test T2,T3;
  cout<<"leaving block\n";

  }
  cout<<"back inside main\n";
  return 0;
 }
```

```
#include <iostream>
using namespace std;
int count=0;
class test
{

   public:
     test( )
     {
      count++;
            cout<<" constructor for object
number"<<count<<"\n";

     }
     ~test()
     {
     cout<<"destructor for object number"<<count<<endl;
     count--;

     }
 };
```

```
int main()
 {
  cout<<"inside main\n";
  cout<<"creating object T1\n";
  test T1;
  {
  //Block
  cout<<"creating two more object in block
T2 and T3\n";
  test T2,T3;
  cout<<"leaving block\n";

  }
  cout<<"back inside main\n";
  return 0;}
```

inside main
creating object T1
constructor for object number1
creating two more object in block T2 and T3
constructor for object number2
constructor for object number3
leaving block
destructor for object number3
destructor for object number2
back inside main
destructor for object number1

```
#include<iostream>

class Test{
	int *ip;
public:
	Test(int i = 0) {
		ip = new int;
		*ip = i;
		std::cout << "Constructor called ";
		std::cout << *ip << std::endl;
	}
	void print() {
		std::cout << *ip << std::endl;
	}
	~Test() {
		std::cout << "Destructor called ";
		std::cout << *ip << std::endl;
		delete ip;
	}
};
```

```
Test gt;

int main() {

	std::cout << "Inside main
function\n";

	Test lt(9);
	Test &dt = *(new Test(5));

	gt.print();
	lt.print();
	dt.print();

	delete &dt;
	std::cout << "main is about to
end\n";
	return 0;
}
```

```
Test &dt = *(new Test(5));
```

- The * (asterisk) is the dereference operator.
- It is used to access (yield) the actual object that a pointer refers to (not the pointer itself).

Why is * used?

- `new Test(5)` creates a Test object in heap memory and returns a pointer to it.
- Using * in `*(new Test(5))` dereferences the pointer, giving the actual object instead of the pointer.
- We need the object itself for the reference binding, not the pointer.

```
#include<iostream>

class Test{
	int *ip;
public:
	Test(int i = 0) {
		ip = new int;
		*ip = i;
		std::cout << "Constructor called
";
		std::cout << *ip << std::endl;
	}
	void print() {
		std::cout << *ip << std::endl;
	}
	~Test() {
		std::cout << "Destructor called ";
		std::cout << *ip << std::endl;
		delete ip;
	}
};
Test gt;

int main() {

	std::cout << "Inside main function\n";

	Test lt(9);
	Test &dt = *(new Test(5));

	gt.print();
	lt.print();
	dt.print();

	delete &dt;
	std::cout << "main is about to end\n";
	return 0;
}
```

```
Constructor called 0
Inside main function
Constructor called 9
Constructor called 5
0
9
5
Destructor called 5
main is about to end
Destructor called 9
Destructor called 0
```

const object and const method

```
#include<iostream>
#include<cstring>
using std::cout;
using std::endl;

class Str {
    int len;
    char *strng;a
public:
    void print() {
        cout << len << ": ";
        puts(strng);
    }

    Str(const char *strng) {
        len = strlen(strng);
        this->strng = new char[len + 1];
        strcpy(this->strng, strng);
        cout << "constructor: " << strng << endl;
    }

    ~Str() {
        cout << "Destructor: " << strng << endl;
        delete strng;       }
};
```

```
void fun(Str &ln) {
    ln.print();
    cout << "Before deleting dynamic
object\n";
    delete &ln;
    cout << "After deleting dynamic
object\n";
}

int main() {
    Str first_name("Chintan");
    const Str middle_name("Ravi");
    Str *last_name = new
Str("Chatterjee");

    first_name.print();
    middle_name.print();
    fun(*last_name);
    last_name->print();

    return 0;
}
```

const object and const method

```
#include<iostream>
#include<cstring>
using std::cout;
using std::endl;

class Str {
    int len;
    char *strng;
public:
    void print() {
        cout << len << ": ";
        puts(strng);
    }

    Str(const char *strng) {
        len = strlen(strng);
        this->strng = new char[len + 1];
        strcpy(this->strng, strng);
        cout << "constructor: " << strng << endl;
    }

    ~Str() {
        cout << "Destructor: " << strng << endl;
        delete strng;   // ❌ ERROR (should be delete[])
    }
};
```

```
void fun(Str &ln) {
    ln.print();
    cout << "Before deleting dynamic object\n";
    delete &ln;   // ❌ ERROR (deleting reference to
stack object)
    cout << "After deleting dynamic object\n";
}

int main() {
    Str first_name("Chintan");
    const Str middle_name("Ravi");
    Str *last_name = new Str("Chatterjee");

    first_name.print();
    middle_name.print();  // ❌ ERROR (const object
calling non-const method)

    fun(*last_name);      // ❌ runtime error: double
delete
    last_name->print();   // ❌ segmentation fault

    return 0;
}
```

const object and const method

- **Data members (private or public) of const object can not be changed. Neither by member function nor from outside the object**
- const objects can call only const member functions
- **Const member functions have const keyword after prototype (during definition)**
- const member functions can not change any data member of the object (even when object is non-const)
- **Const methods can have auto variables and they can change**
- const member functions can be invoked by non-const objects
- **When method is not changing any data members of the object, it is good practice to declare it const**

Destructors

```
#include<iostream>
#include<cstring>
using std::cout;
using std::endl;
class Str
{
   int len;
   char *strng;
public:
   void print() const
   {
      cout << len << ": ";
      puts(strng);
   }
   Str(const char *strng)
   {
      len = strlen(strng);
      this->strng = new char[len + 1];
      strcpy(this->strng, strng);
      cout << "constructor: " << strng << endl;
   }
   ~Str()
   {
      cout << "Destructor: " << strng << endl;
      delete strng;
   }};
```

```
void fun(Str &ln)
{
   ln.print();
   cout << "Before deleting dynamic
object\n";
   delete &ln;
   cout << "After deleting dynamic object\n";
}

int main()
{
   Str first_name(“Chintan");
   const Str middle_name(“Ravi");
   Str *last_name = new Str(“Chatterjee");
   first_name.print();
   middle_name.print();
   fun(*last_name);
   return 0;
}
```

Destructors

```
#include<iostream>
#include<cstring>
using std::cout;
using std::endl;
class Str
{
   int len;
   char *strng;
public:
   void print() const
   {
      cout << len << ": ";
      puts(strng);
   }
   Str(const char *strng)
   {
      len = strlen(strng);
      this->strng = new char[len + 1];
      strcpy(this->strng, strng);
      cout << "constructor: " << strng << endl;
   }
   ~Str()
   {
      cout << "Destructor: " << strng << endl;
      delete strng;
   }};
```

```
void fun(Str &ln)
{
   ln.print();
   cout << "Before deleting dynamic
object\n";
   delete &ln;
   cout << "After deleting dynamic object\n";
}

int main()
{
   Str first_name("Chintan");
   const Str middle_name("Ravi");
   Str *last_name = new Str("Chatterjee");
   first_name.print();
   middle_name.print();
   fun(*last_name);
   return 0;
}
```

```
constructor: Chintan
constructor: Ravi
constructor: Chatterjee
7: Chintan
4: Ravi
10: Chatterjee
Before deleting dynamic object
Destructor: Chatterjee
After deleting dynamic object
Destructor: Ravi
Destructor: Chintan
```

Destructors

- **If we do not release dynamic memory allocated for the object's data members, then it will lead to memory leaks**
  - **Once an object's lifetime ends, its destructor will be called.**
  - **After execution of the destructor, the object's memory would be freed.**
  - **If dynamic memory allocated for data members has not been freed already and if we do not free it in destructor, then it can not be freed until the end of program execution.**
    - **Because we will lose the pointer to dynamic memory allocated for the data member, as the pointer is stored in the data member of the object.**
- **If the object itself has been created dynamically, on calling delete for that object, destructor would be called for it**
  - **Object can be created dynamically (with new keyword) in one function and can be deleted (with delete keyword) in another function.**
  - **The lifetime of a dynamic object is in the developer's control.**
  - **Once a dynamic object is deleted, accessing it is an undefined behaviour and it must be avoided.**